

Security Assessment of a Digital Locker

Prepared By:
Ananth

Part 1 – Threat Modeling

Scope & assumptions:

- ✓ Devices run a Linux-based edge node with local UI, sensors/actuators, MQTT & HTTPS to cloud, and container based OTA.
- ✓ Devices are globally deployed and may be physically accessible to attackers.
- ✓ Cloud provides APIs, device management, OTA distribution, and telemetry ingestion.

Threat Model Summary Table

1. Edge Device (Physical & Software Threats)

Threat Category	Attack Scenario	Impact	Mitigation
Tampering	Attacker gains physical access and connects debugging probes (e.g, JTAG, UART) to extract firmware or cryptographic keys	Confidentiality and Integrity loss. Device compromise, fleet keys exposed	Disable Debug Ports in production devices. Use Hardware Security Module (HSM) or Trusted Platform Module (TPM) for key storage. And implement Secure Boot.
Spoofing	Attacker manages to bypass the local authentication mechanism (e.g, weak password, session fixation) to impersonate a legitimate user or administrator.	Integrity loss, unauthorized local command execution	Enforce strong, complex local passwords. Implement session timeouts and strong CSRF/XSS protection on the web interface.
Denial of Service	Flooding services, exhausting CPU or blocking network	Availability of the service will be impacted	Rate limiting, watchdogs, quotas, network filtering, hardware watchdog recovery

Elevation of Privilege	A vulnerability (e.g, buffer overflow, insecure deserialization) in the local web or touchscreen interface allows a non-privileged user to execute code as root	Integrity and Confidentiality loss. Full device takeover	Use of SELinux or AppArmor policies to confine the process and limit syscalls. Input validation and secure coding practices. Follow the principle of least privilege
Information Disclosure	Extraction of credentials or telemetry from local storage or memory	Confidentiality of sensitive data	Store secrets in hardware keystore, encrypted storage, short-lived credentials, secure memory handling.

2. Communication Channel (Device <---> Cloud)

Threat Category	Attack Scenario	Impact	Mitigation
Tampering & Information Disclosure	An attacker performs a passive network tap or Man in the Middle (MITM) attack to intercept MQTT telemetry or API responses	Confidentiality loss of sensor data, device location, and cloud configuration details	Mutual TLS (mTLS) for all device-cloud communication. Enforce TLS 1.3 and strong ciphers
Spoofing	An attacker replicates the device's unique identifier and tries to connect to the cloud, injecting false data or commands	Integrity loss of cloud data, potential loss of Availability if the cloud platform acts on false commands.	Mutual Authentication: Cloud only accepts connections from devices presenting a valid, non-revoked, unique Client Certificate issued by the trusted PKI
Repudiation	A compromised device sends a malicious command (e.g., open a vault), and later the attacker denies the device was the source of the command.	Integrity loss of audit trail; difficulty in forensic investigation.	Non-Repudiation enforced by the mTLS Client Certificate: The private key proves the command originated from that specific, cryptographically verified device identity. All commands are logged with the device ID
Denial of Service	An attacker floods the MQTT broker or HTTPS API gateway with a massive volume of connection requests	Unable to process legitimate traffic from the device fleet	Rate limiting based on device identity, source IP, and connection frequency

3. Backend / Cloud Infrastructure

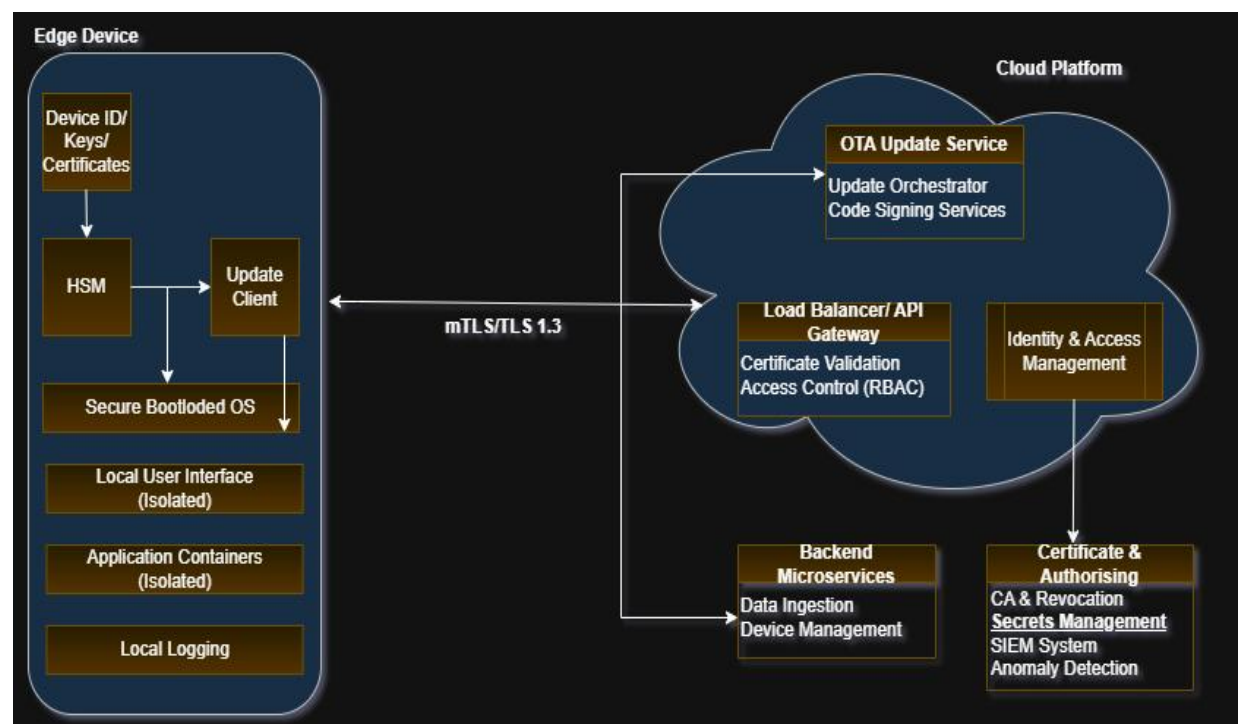
Threat Category	Attack Scenario	Impact	Mitigation
Denial of Service	A large number of compromised devices (botnet) or a single misconfigured device flood the cloud API with excessive requests	Availability loss for the entire device fleet and backend services	Rate Limiting based on device ID and API endpoint. Deploy DDoS Protection (WAF/Cloud Native Protection). Implement a scalable, queue-based architecture for high-volume inputs
Tampering	An attacker compromises the update signing server and injects malicious code into a container update image	Integrity loss, mass device compromise, or device bricking (DoS)	Air-Gapped Root Signing Key and strong key rotation policies. Devices must perform cryptographic signature verification of the update manifest and its associated binaries against the trusted Root CA/keys before installation
Information Disclosure	An attacker captures the update manifest or image during transmission, exposing the proprietary software design and features	Loss of data confidentiality	Encrypting the OTA Payloads (on top of mTLS transport) is ideal for maximum IP protection, though signature verification is the priority integrity control. Access control to the update artifacts is strictly limited.
Elevation of Privilege	A flaw in the cloud API's authorization logic allows one device identity to issue commands intended for another device	Integrity loss. Device-to-device attack potential	Strict Authorization Checks: All API calls must enforce Principle of Least Privilege

Part 2 - Secure Architecture Proposal

Objectives:

- ✓ Provide strong, hardware-rooted device identity and mutual authentication.
- ✓ Protect the update channel and secrets on-device.
- ✓ Provide monitoring and secure operations without sacrificing usability or scalability.

Following below is the proposed secure architecture:



1. Device Identity and Provisioning (Secure Onboarding)

Hardware Root of Trust: Every device is equipped with a Hardware Security Module (HSM). During manufacturing, the HSM is provisioned with a unique, immutable Initial Client Certificate (ICC) and the corresponding private key, which is non-exportable.

Zero-Touch Provisioning (ZTP): On first boot, the device connects to the cloud's provisioning service over mTLS using its ICC. The provisioning service validates the ICC and performs an optional remote attestation check of the device's boot state (if supported by the TPM).

Operational Certificate Issuance: If attestation and validation succeed, the cloud issues a short-lived operational certificate (e.g, 90-day lifespan) to the device. This certificate is stored securely in the HSM and used for all ongoing communication.

2. Mutual Authentication (mTLS)

Mandatory mTLS: All communication between the Edge Device and the Cloud API Gateway is secured using Mutual TLS (mTLS) 1.3.

Device --> Cloud: The device presents its operational certificate (client certificate). The cloud verifies the certificate's authenticity against its trusted PKI chain and checks the Certificate Revocation List (CRL) or OCSP status via the Identity & Management Service.

Cloud --> Device: The Cloud API Gateway presents its server certificate, which the device validates against a pinned Root CA public key stored in its Secure Boot ROM or HSM. This prevents the device from connecting to a malicious or compromised cloud endpoint.

3. Update Security (OTA)

Code Signing Hierarchy: All Over the Air (OTA) updates are cryptographically signed using a hierarchical key system. The top level master key is stored in an air-gapped, offline HSM (Code Signing Service) within the cloud backend.

Integrity and Authenticity: The device's Update Client downloads the update manifest and performs two critical checks before installation:

- ✓ **Signature Verification:** Verifies the update's signature chain against the trusted public key stored in the device's secure storage.
- ✓ **Manifest Hash Check:** Verifies the cryptographic hash of the container image inside the manifest.

Rollback Protection: An increasing version number is stored in the HSM. The Update Client will refuse to install any update with a version number lower than the currently enforced version, preventing attackers from forcing the device back to a vulnerable state.

4. Secrets Management

On Device: Critical secrets (device private keys, local symmetric keys) are stored and operated inside the HSM and are never exposed to the general operating system. Non critical runtime secrets are encrypted using a key derived from the HSM and stored in isolated memory.

On Cloud: A dedicated, centralized Secrets Management solution (e.g, a Key Vault) is used to store all cloud-side secrets (database credentials, API keys for external services). Access is restricted via strict IAM policies adhering to the principle of least privilege.

Rotation Policy: All operational mTLS certificates are automatically rotated on a schedule (e.g, quarterly) well before expiration. Cloud service API keys are rotated frequently, enforcing a "break-glass" procedure for manual intervention.

5. Monitoring & Logging

Event Telemetry: The Edge Device performs local logging of security-critical events (e.g, failed updates, secure boot warnings, abnormal process terminations) and sends this telemetry via mTLS to the cloud.

Centralized SIEM: All logs from the device, the API Gateway (mTLS failures, unauthorized calls), the OTA Update Service (signature validation failures), and the Cloud IAM are ingested into a Security Information and Event Management (SIEM) system.

Anomaly Detection: An analytical layer within the SIEM establishes baseline behavioral models (e.g, expected data volume, connection patterns, geographical location). The system generates high priority alerts for deviations, such as a device abruptly increasing its data rate or a failed attempt to use a revoked certificate, which enables proactive incident detection.

Part 3 – Security Governance & Compliance

Ensuring "Security by Design"

The "Security by Design" approach should be embedded through a DevSecOps model, by integrating security activities throughout the Software Development Lifecycle (SDLC).

Threat Modeling: must be a mandatory step at the design phase of every new feature, service, or major architectural change (as demonstrated in Part 1). This shifts security left, preventing vulnerabilities before they are coded.

Security Gates in CI/CD: Implement automated security checks in the Continuous Integration/Continuous Deployment (CI/CD) pipeline:

- ✓ Static Application Security Testing (SAST) for code quality
- ✓ Software Composition Analysis (SCA) to identify vulnerabilities in open source dependencies.
- ✓ Container Scanning to check base images for OS/library vulnerabilities.

Secure Coding Standards: Define and enforce language specific secure coding standards and playbooks for common vulnerabilities (e.g, input validation, secure use of cryptographic primitives).

Alignment to IoT Security Standards and Frameworks

The system will align primarily with the following standards, prioritizing those that cover both device and ecosystem security:

ETSI EN 303 645 (Cyber Security for Consumer IoT): This is the primary alignment target as it provides a practical baseline of 13 high level provisions covering authentication, vulnerability management, security by design, and communication security. It is widely adopted and becoming a legislative baseline globally.

NISTIR 8259A (IoT Device Cybersecurity Capability Core Baseline): This framework provides a strong technical foundation, especially for device

integrity, identity, and access control. It aligns well with the proposed use of HSM and secure boot.

OWASP IoT Top 10: This can be used as a practical tool for penetration testing scope and developer education, focusing on common implementation flaws like weak passwords, insecure updates, and insufficient privacy controls.

Vulnerability Disclosure & Patch Management

Vulnerability Disclosure Policy (VDP): Maintain a clear, public policy to facilitate responsible reporting by external security researchers.

Prioritized Patching: Vulnerabilities will be triaged using CVSS scoring. Critical patches will be deployed immediately via the highly secure OTA channel, leveraging the architecture's mandatory update enforcement and rollback protection.

Incident Response (IR)

IR Team and Playbooks: Establish a dedicated, cross-functional IRT and develop playbooks for scenarios like device botnets or cloud key compromise.

Detect, Contain, Eradicate: Our primary goal is rapid response. The SIEM/Anomaly Detection system will trigger alerts, leading to immediate containment (e.g, revoking the compromised devices mTLS certificate or isolating the affected cloud environment) before remediation.

Every incident requires a thorough review to update security controls and prevent recurrence.